PYTHON TASK

1.TASK

```
MIN_FAT = 2.0
MAX_FAT = 7.0
DAYS = 10

farmer_id = int(input("Enter Farmer ID: "))
litres = float(input("Enter litres: "))
fat = float(input("Enter fat percentage: "))
snf = float(input("Enter SNF percentage: "))
shift = input("Enter shift: ")

if farmer_id <= 0:
    print("Invalid Farmer ID")
elif litres < 0:
    print("Invalid milk quantity")
elif fat < MIN_FAT or fat > MAX_FAT:
    print("Invalid fat percentage")
elif snf < 0:
    print("Invalid SNF percentage")
elif shift.lower() not in ["morning", "evening"]:
    print("Invalid shift")
else:
    rate_per_litre = float(input("Enter rate per litre: "))

    if rate_per_litre < 0:
```

```
    print("Invalid rate")
else:
    amount = litres * rate_per_litre
    print("Rate per litre:", rate_per_litre)
    print("Payment amount:", round(amount, 2))

    ten_day_total = 0.0

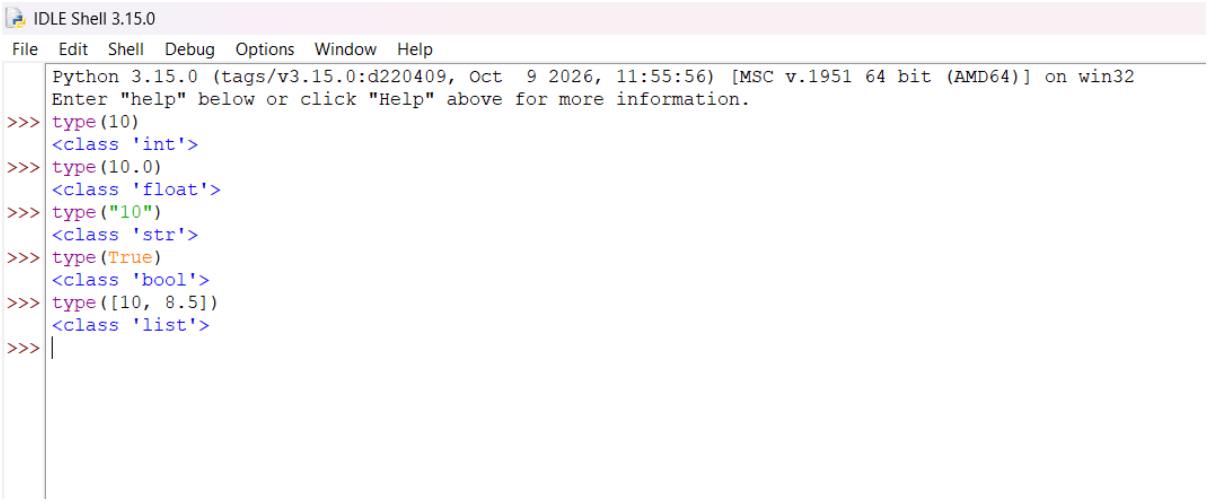
    for day in range(DAYS):
        payment = float(input("Enter payment for day " + str(day + 1) + ": "))
        if payment < 0:
            print("Invalid payment")
            break
        ten_day_total += payment
    else:
        print("Ten-day total:", round(ten_day_total, 2))
```

Output

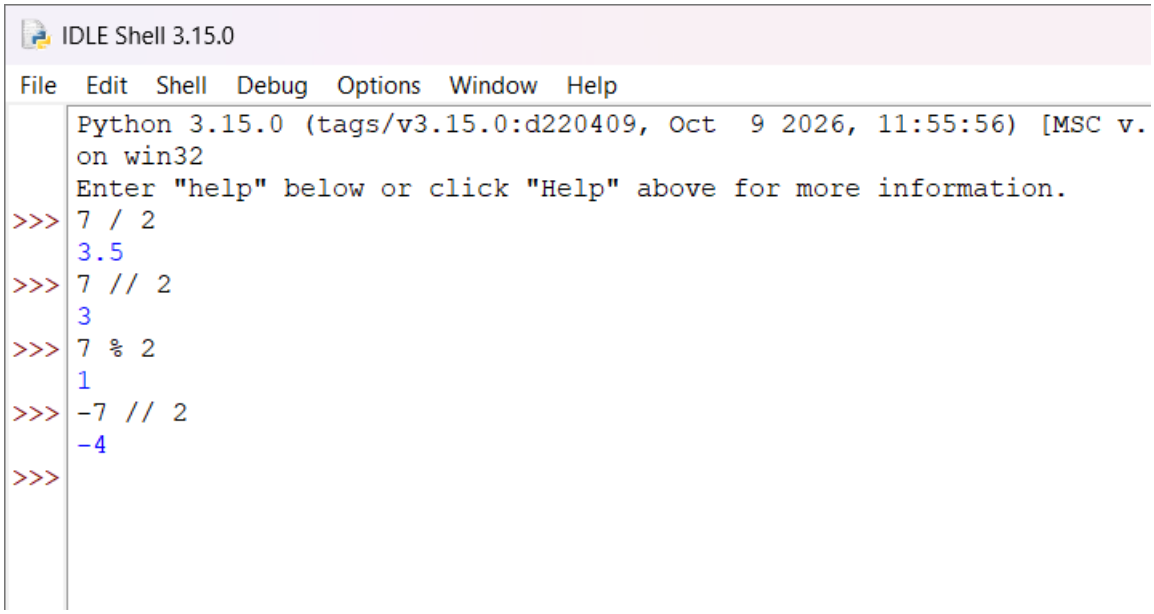
```
Enter Farmer ID: 101
Enter litres: 20
Enter fat percentage: 4.5
Enter SNF percentage: 8.5
Enter shift: morning
Enter rate per litre: 40
Rate per litre: 40.0
Payment amount: 800.0
Enter payment for day 1: 800
Enter payment for day 2: 720
Enter payment for day 3: 850
Enter payment for day 4: 820
Enter payment for day 5: 790
Enter payment for day 6: 800
Enter payment for day 7: 777
Enter payment for day 8: 780
Enter payment for day 9: 840
Enter payment for day 10: 800
Ten-day total: 7977.0
```

2.TASK

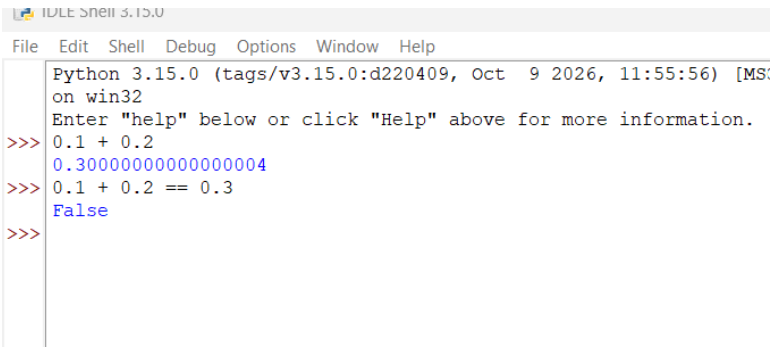
i)



ii)



iii)



Explanation: Python cannot represent some decimal numbers exactly in binary, causing small precision errors.

iv)

```
from decimal import Decimal


payments = ["33.40"] * 1000 + ["0.10"] * 3


def total_float(items):
    total = 0.0
    for amount in items:
        total += float(amount)
    return total


def total_decimal(items):
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
    return total


print("float :", total_float(payments))
print("Decimal:", total_decimal(payments))
print(0.1 + 0.2 == 0.3,
      Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
```

```
>>
============= RESTART: C:/Users/Murtaza Malik Khan/OneDrive/Docu
float : 33400.30000000061
Decimal: 33400.30
False True
>>
```

Explanation: float produces a small precision error, while Decimal calculates the exact money total.

V)

```
>>> from decimal import Decimal
>>> Decimal(float("0.1"))
Decimal('0.1000000000000000055511151231257827021181583404541015625')
>>>
```

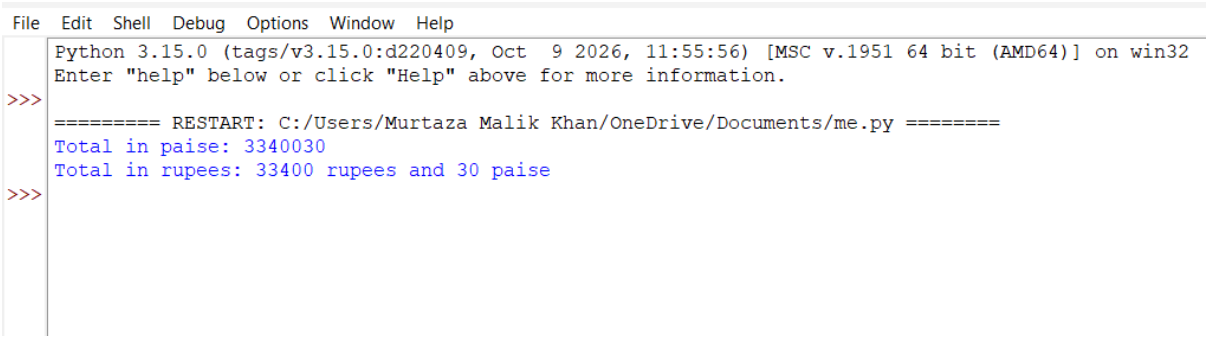
Explanation: Converting to float first introduces a precision error. Use Decimal directly from strings for accurate results.

Vi)

```
payments = ["33.40"] * 1000 + ["0.10"] * 3


def total_paise(items):
    total = 0
    for amount in items:
        rupees, paise = amount.split(".")
        total += int(rupees) * 100 + int(paise)
    return total


print("Total in paise:", total_paise(payments))
print("Total in rupees:", total_paise(payments) // 100,
      "rupees and", total_paise(payments) % 100, "paise")
```



Explanation: Each payment is converted into integer paise before addition, avoiding floating-point precision errors.

3.TASK

1. Naming Rules and Conventions

- Variable names should start with a letter or underscore.
- Variable names cannot contain spaces or Python keywords.
- Use lowercase letters for variables, such as milk_rate.
- Use uppercase letters for constants, such as BASE_RATE.
- Use meaningful names to make the program easy to understand.

2.Why Python Has No True Constants

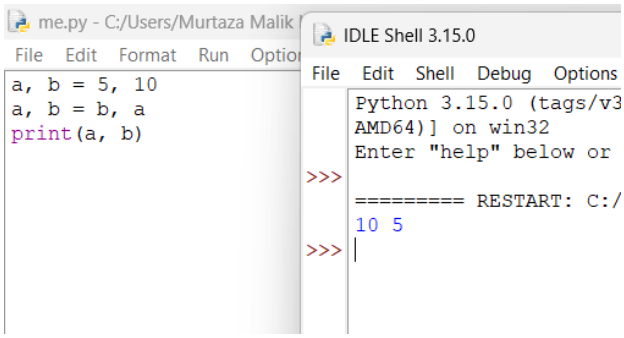
- Python does not have built-in constant variables that cannot be changed. Instead, uppercase names such as `BASE_RATE` and `FAT_RATE` indicate that the values should not be changed.
- If a milk rate changes, we only need to update its constant in one place.

3. Dynamic Typing

- Python allows the same variable to store different types of values.
- `x = 10x = "Hello"`
- Initially, `x` is an integer. Later, it becomes a string. Python automatically determines the type.

4. Multiple Assignment and Swapping

- Python allows multiple variables to be assigned in one statement.



5. BASE_RATE = 30.0

FAT_RATE = 0.40

SNF_RATE = 0.30

MIN_FAT = 2.0

MAX_FAT = 7.0

litres = 10

fat = 4.0

snf = 8.0

amount = litres * (BASE_RATE + fat * FAT_RATE + snf * SNF_RATE)

print("Milk quantity:", litres)

print("Milk rate:", BASE_RATE)

print("Payment amount:", round(amount, 2))

value = 10

print(type(value))

value = "Hello"

print(type(value))

a, b = 5, 10

a, b = b, a

print("a =", a)

print("b =", b)

```
========= RESTART: C:/Users/Murtaza Malik Khan/OneDrive/Documents/me.py ===
Milk quantity: 10
Milk rate: 30.0
Payment amount: 340.0
<class 'int'>
<class 'str'>
a = 10
b = 5
>>>
```

Rates can be stored in a dictionary:

RATE_CARD = {"base": 30.0,"fat": 0.40,"snf": 0.30}

4.TASK

i)

me.py - C:/Users/Murtaza Malik Khan/OneDrive/Documents/me.py (3.15.0)

File Edit Format Run Options Window Help

```
import math

MIN_FAT = 2.0
MAX_FAT = 7.0
FAT_STEP = 0.1
BASE_RATE = 30.0
FAT_RATE = 0.40
SNF_RATE = 0.30

fat = 4.2
snf = 8.5
litres = 10

fat_steps = round((fat - MIN_FAT) / FAT_STEP)
snf_steps = round((snf - 7.0) / FAT_STEP)

rate_per_litre = BASE_RATE + fat_steps * FAT_RATE + snf_steps * SNF_RATE
amount = litres * rate_per_litre

print("Fat steps:", fat_steps)
print("SNF steps:", snf_steps)
print("Rate per litre:", round(rate_per_litre, 2))
print("Total amount:", round(amount, 2))
```

IDLE Shell 3.15.0

File Edit Shell Debug Options Window Help

```
Python 3.15.0 (tags/v3.15.0:d220409, Oct  9 2026, 11:5
AMD64)] on win32
Enter "help" below or click "Help" above for more infor
>>>
========= RESTART: C:/Users/Murtaza Malik Khan/OneDrive
Fat steps: 22
SNF steps: 15
Rate per litre: 43.3
Total amount: 433.0
>>>
```

ii)Operator Precedence and Associativity

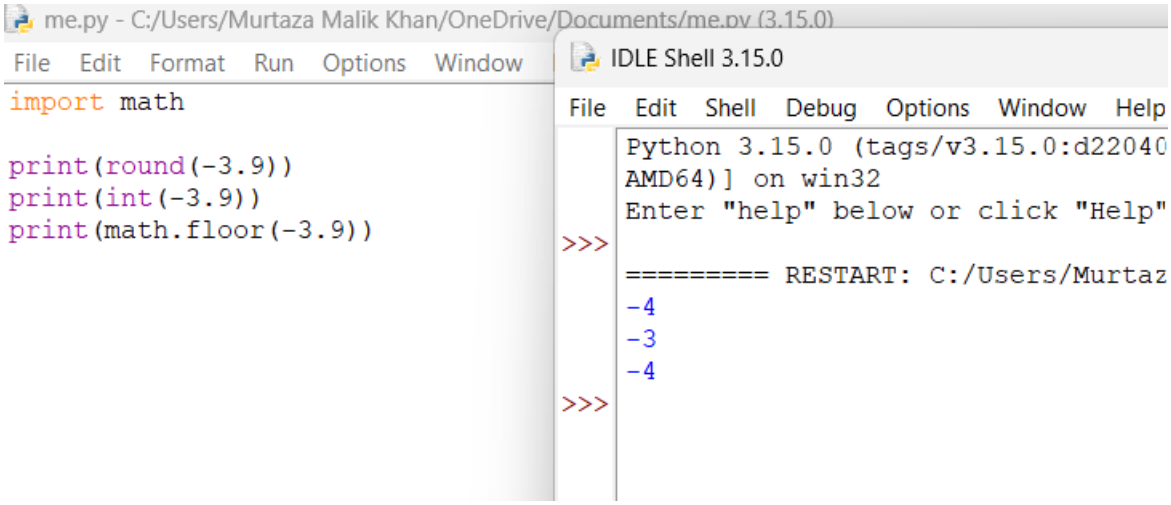
- ** means exponentiation and is evaluated before multiplication and addition.
- * and / are evaluated before + and -.
- Parentheses () are evaluated first.
- Unary minus changes the sign of a value. For example, -2**2 gives -4, because exponentiation happens before unary minus.
- Comparisons can be chained, for example, 2.0 <= fat <= 7.0.
- Boolean operators such as and, or, and not combine or reverse logical conditions.

iii) Floating-Point Error in Step Calculation

```
(4.2 - 3.5) / 0.1
7.000000000000002
round((4.2 - 3.5) / 0.1)
7
```

Explanation: Floating-point numbers cannot always represent decimal values exactly. Using round() gives the expected step count of 7.

iv) Difference Between round(), int() and math.floor()

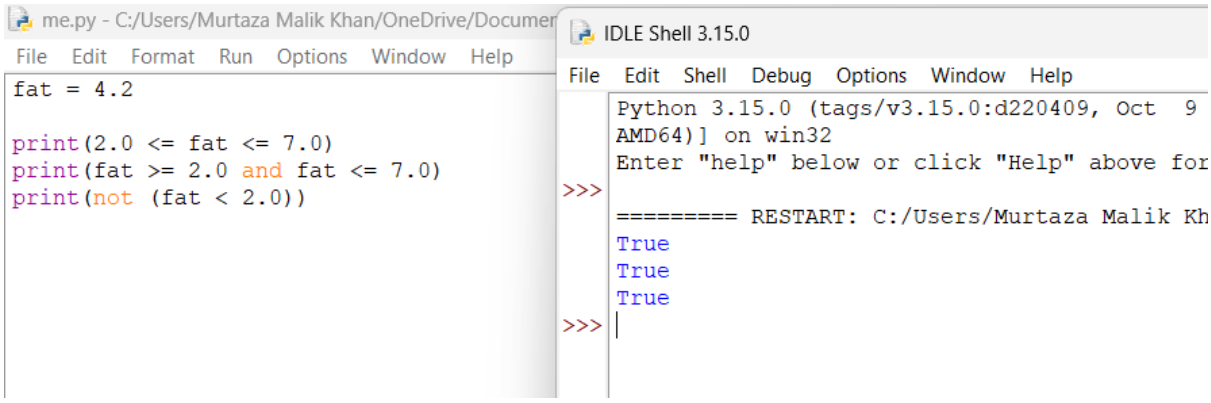


round() rounds to the nearest integer.

int() removes the decimal part, moving towards zero.

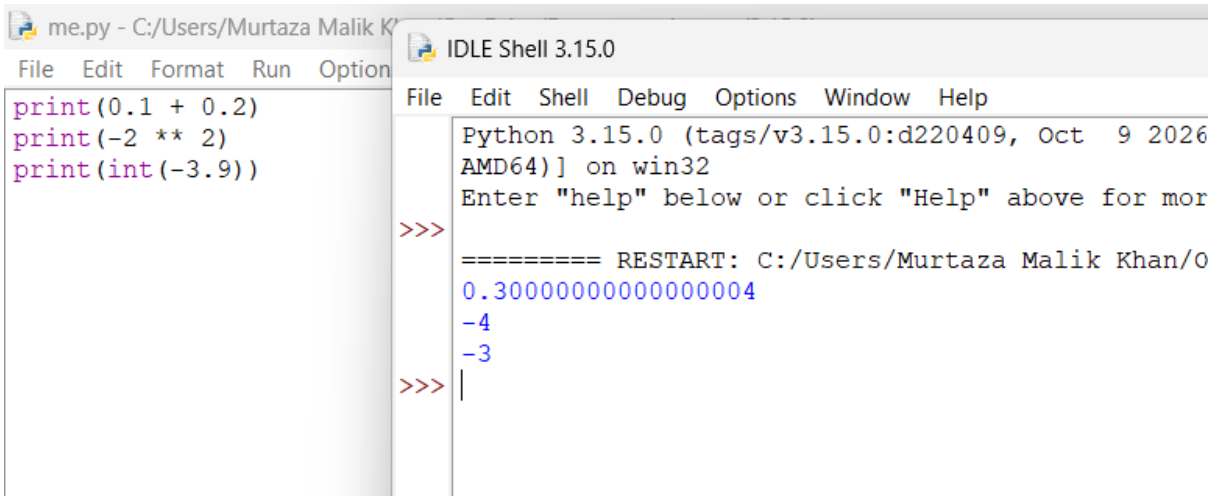
math.floor() rounds down towards negative infinity.

V) Comparison Chaining and Boolean Operators



Comparison chaining checks whether the fat value lies within the specified range. The and operator combines conditions, while not reverses a Boolean result.

vi) Three Unexpected Results



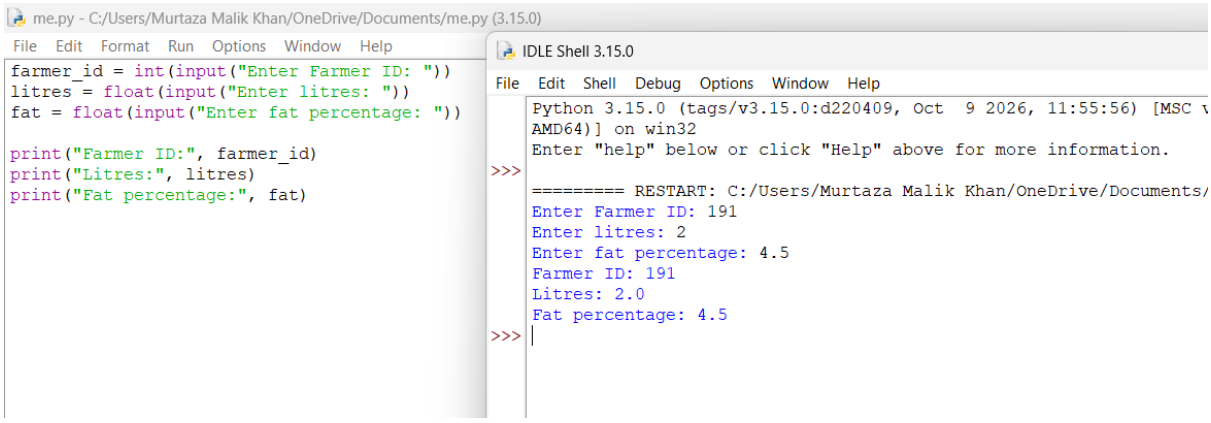
Explanation:

1. 0.1 + 0.2 produces a small precision error because of binary floating-point representation.
2. -2 ** 2 gives -4 because exponentiation occurs before unary minus.
3. int(-3.9) gives -3 because int() truncates towards zero.

5.TASK

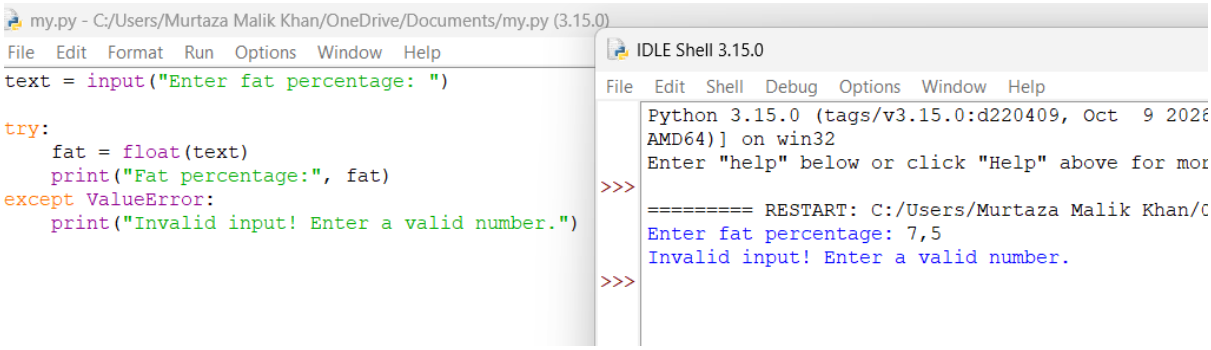
i) Reading Input and Converting Types

In Python, input() always returns a string. We use int() to convert input into an integer and float() to convert it into a decimal number.

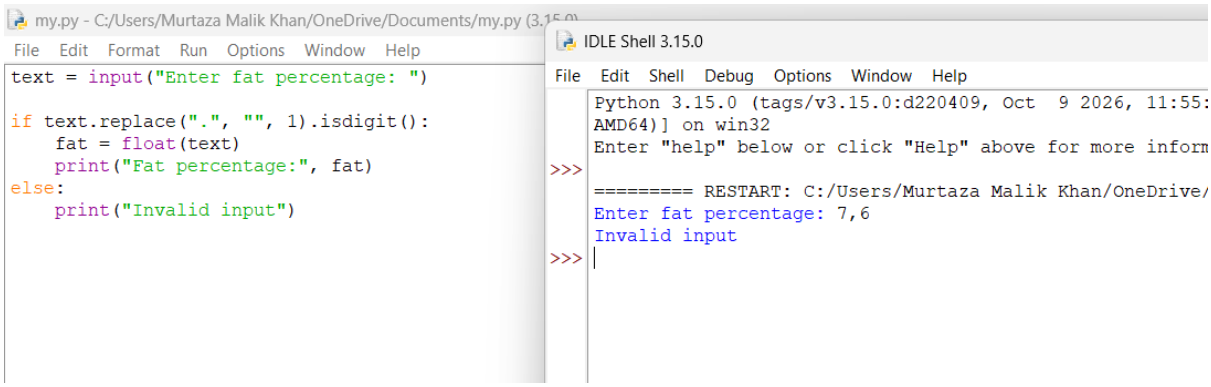


ii) Handling Invalid Input

If the clerk enters "8,5" or "eight" instead of 8.5, float() raises a ValueError:

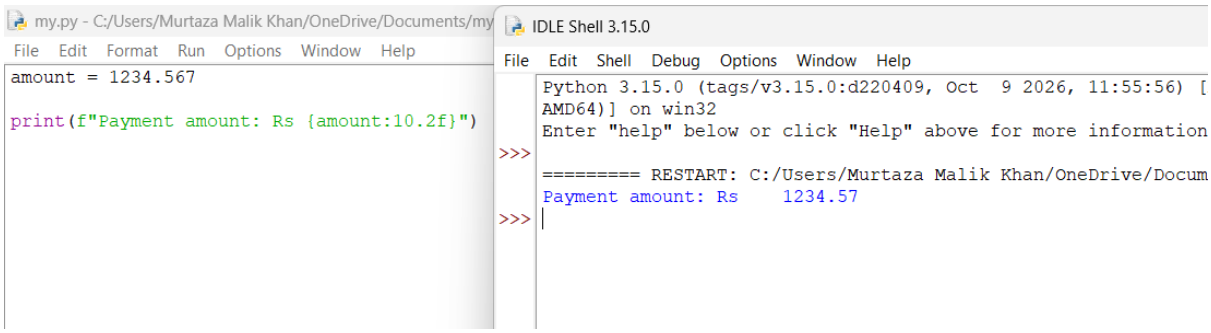


iii) Checking Input Before Conversion

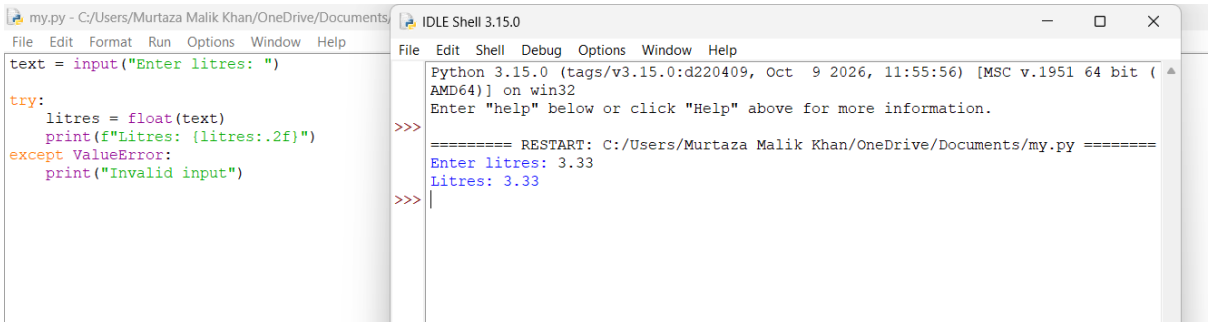


This checks whether the input contains digits and at most one decimal point. However, it does not handle negative numbers. Using try/except ValueError is generally more reliable.

iv) Formatting Output Using f-Strings

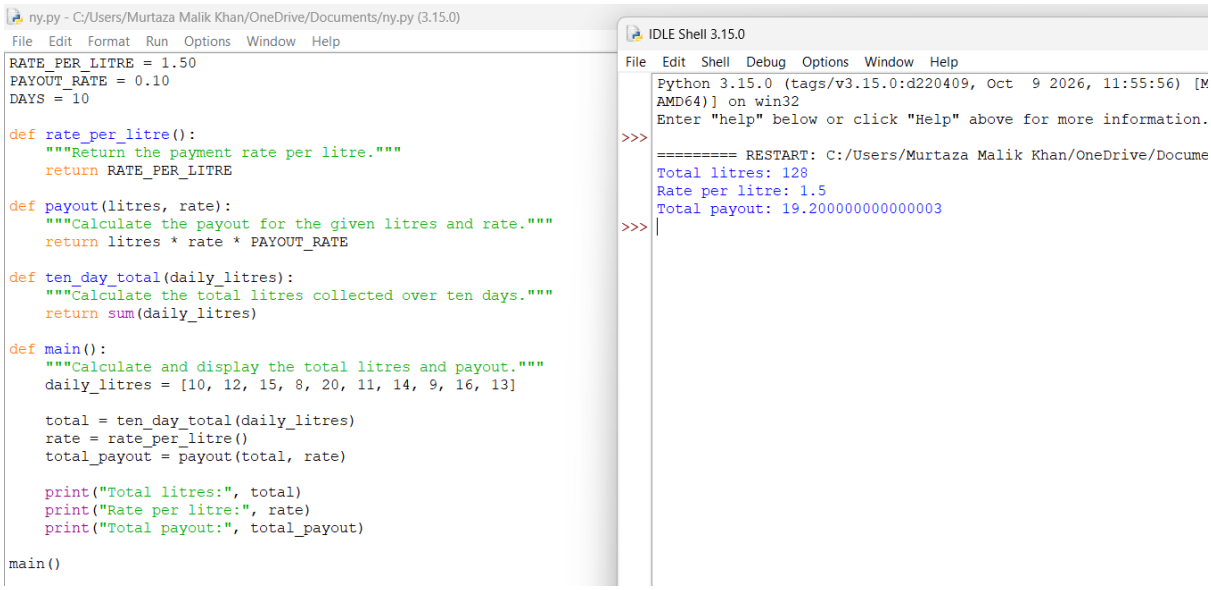


v) Alternative Method and Complexity Analysis



6. TASK

Organise the Logic into Functions



Explanation

1. Functions, parameters and return values: rate_per_litre() returns the rate, payout(litres, rate) accepts litres and rate and returns the payout, and ten_day_total(daily_litres) accepts a list and returns its sum.
2. Returning vs printing: return sends a value back to the calling code so it can be reused. print() only displays the value on the screen.
3. Local and global variables: RATE_PER_LITRE, PAYOUT_RATE and DAYS are global constants. They can be read inside functions but should not be changed. Variables such as total and rate inside main() are local variables.
4. Docstrings: Each function contains a docstring explaining its purpose.

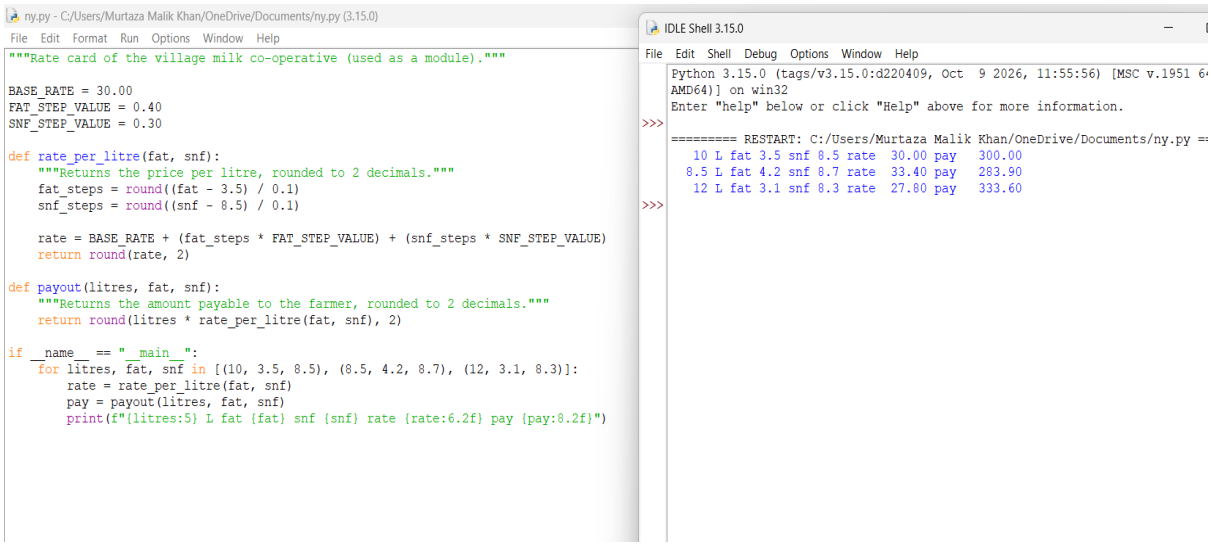
Alternative Concept and Complexity Analysis

Without functions, the same calculations would be written in one long script, causing repeated code and making debugging and maintenance difficult. Lambda expressions are suitable for small, simple operations but are less readable for complex calculations.

For n entries, calculating the total takes O(n) time. Function calls introduce a small overhead, but this is generally negligible for ordinary datasets. The solution uses (O(n)) space for the list of daily values. Organising the logic into functions improves readability, reusability and testing.

7.TASK

i) milk_rate.py



ii) test_milk.py

```
import milk_rate


print(milk_rate.rate_per_litre(4.2, 8.7))
print(milk_rate.payout(8.5, 4.2, 8.7))
```

output:

```
33.40
283.90
```

iii) Explanation

- import milk_rate imports the entire module. Functions and constants are accessed using milk_rate..
- from milk_rate import payout imports only the payout() function.
- import milk_rate as mr imports the module using the shorter alias mr.
- if __name__ == "__main__": ensures the example table runs only when the module is executed directly. When another script imports the module, the table is not printed.
- Standard modules: math provides mathematical functions, while decimal supports precise decimal arithmetic for financial calculations.
- Module search path: Python searches the current script's directory and locations listed in sys.path to find modules.

iv) Alternative Concept and Complexity Analysis

If the functions were copied into every script, the same code would be repeated. Whenever the milk rate changes, every copy would need to be updated, increasing maintenance effort and the chance of errors.

Using a module keeps the functions in one place, so changes need to be made only once. Python loads an imported module once per interpreter session and caches it in sys.modules, avoiding repeated loading for subsequent imports.

The calculations take O(1) time per entry. Processing n entries takes O(n) time. Importing a module adds a small one-time overhead, but reusing the module improves maintainability and avoids duplicated code.

8.TASK

i) Steps of Algorithmic Problem Solving

1. Understand: Calculate the payment for each day using litres, fat, SNF and the milk rate.
2. Plan: Write pseudocode and a flowchart before coding.
3. Write: Convert the pseudocode into Python code.
4. Test: Test the program with different values and check the total payment.

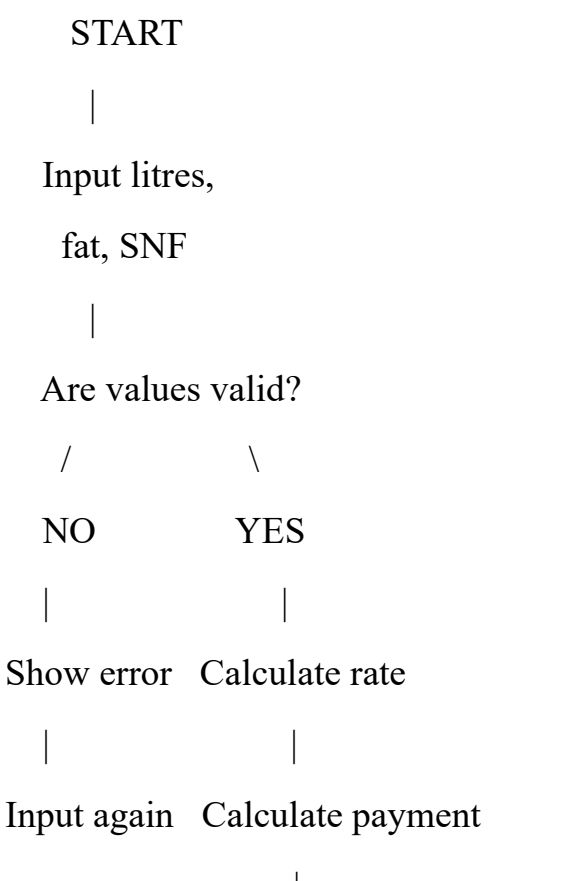
ii) Pseudocode

```
BEGIN
   total = 0

   FOR each of the 10 days
      INPUT litres, fat, snf
      rate = calculate_rate(fat, snf)
      payment = litres * rate
      DISPLAY payment
      total = total + payment
   END FOR

   DISPLAY total
END
```

iii) Flowchart for Validating One Entry



iv) Hand Trace for Three Entries

Day	Litres	Fat	SNF	Rate (Rs.)	Payment (Rs.)	Running Total (Rs.)
1	**10**	**3.5**	**8.5**	**30.00**	**300.00**	**300.00**
2	**8.5**	**4.2**	**8.7**	**33.40**	**283.90**	**583.90**
3	**12**	**3.1**	**8.3**	**27.80**	**333.60**	**917.50**

Total payment for three entries = Rs. 917.50

v) Alternative Concept and Complexity Analysis

Method 1: Store entries in a list

- Store all payments and add them at the end.
- Time complexity: O(n)
- Space complexity: O(n)
- Useful when payment records are needed later.

Method 2: Running total

- Add each payment to the total immediately.
- Time complexity: O(n)
- Space complexity: O(1)
- Uses less memory because it stores only the total.

Both methods take O(n) time. The running-total method is better when only the final ten-day payment is required, while storing entries is useful when individual records must be reviewed later.

9.TASK

i) Python Code

Save this code as milkrate.py

```
BASE_RATE = 30.00

FAT_STEP_VALUE = 0.40

SNF_STEP_VALUE = 0.30


def rate_per_litre(fat, snf):
    """Calculate milk rate."""
    fat_steps = round((fat - 3.5) * 10)
    snf_steps = round((snf - 8.5) * 10)


    rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps *
SNF_STEP_VALUE
    return round(rate, 2)
```

```
def payout(litres, fat, snf):
    """Calculate payment."""
    return round(litres * rate_per_litre(fat, snf), 2)


def test_calculator():
    """Test the calculator."""
    assert rate_per_litre(3.5, 8.5) == 30.00
    assert rate_per_litre(4.2, 8.7) == 33.40
    assert rate_per_litre(3.1, 8.3) == 27.80
    assert payout(10, 3.5, 8.5) == 300.00
    assert payout(8.5, 4.2, 8.7) == 283.90
    print("All tests passed!")


if __name__ == "__main__":
    test_calculator()
```

ii) Test Table

Test case	Litres	Fat (%)	SNF (%)	Expected result	Actual result
Normal	10	3.5	8.5	Rs. 300.00	Rs. 300.00
Fat above standard	10	4.2	8.5	Rs. 328.00	Rs. 328.00
Fat below standard	10	3.1	8.5	Rs. 284.00	Rs. 284.00
SNF above standard	10	3.5	8.7	Rs. 306.00	Rs. 306.00
SNF below standard	10	3.5	8.3	Rs. 294.00	Rs. 294.00
Boundary values	10	3.5	8.5	Rs. 300.00	Rs. 300.00

iii) Invalid Input Tests

Input	**Expected result**
Text instead of litres	Show an error
Negative litres	Reject the input
Fat below the allowed range	Reject the input
Fat above the allowed range	Reject the input

Note: The code above calculates the payment but does not yet validate invalid inputs. Input validation must be added to reject these cases.

iv) Hand Calculation for Five Entries

Entry	**Litres**	**Fat**	**SNF**	**Rate (Rs.)**	**Payment (Rs.)**
1	10	3.5	8.5	30.00	300.00
2	8	4.0	8.5	32.00	256.00
3	12	3.5	8.7	30.60	367.20
4	5	3.0	8.5	28.00	140.00
5	10	3.5	8.3	29.40	294.00
Total	45				1,357.20

v) Alternative Concept and Complexity Analysis

Manual testing: The user enters each test input and checks the output manually. This takes more time, especially when many tests must be repeated.

Automated testing: A test function uses assert statements to compare expected and actual results. All tests can be run again automatically whenever the code changes.

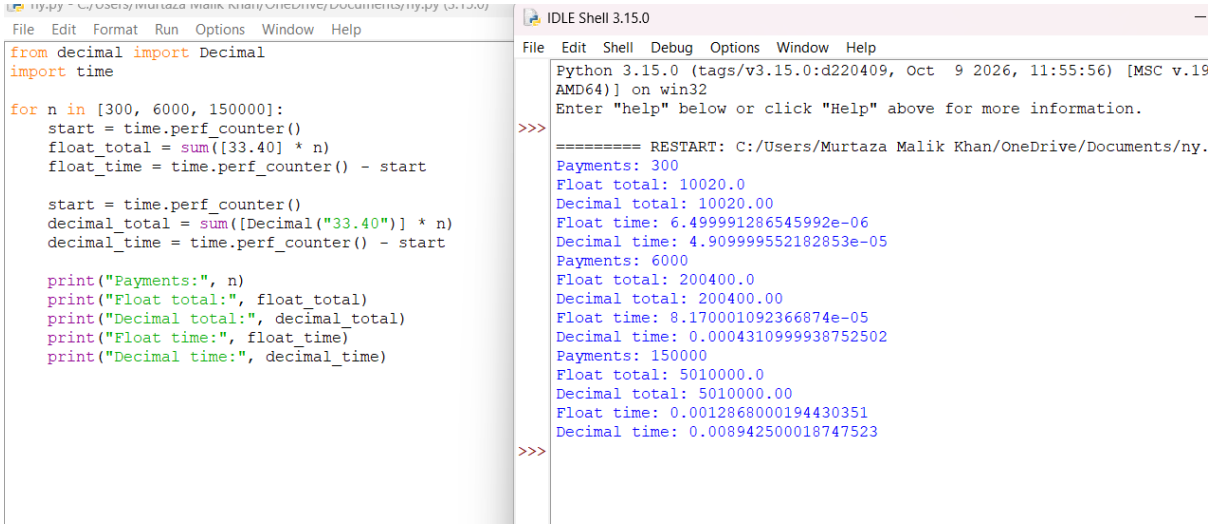
If there are \(n\) test cases, running all tests takes \(O(n)\) time. Automated testing saves time and reduces human errors when the same tests must be repeated.

10. TASK

i) Float and Decimal Comparison

Float values may produce small rounding errors, while Decimal gives more accurate decimal calculations for money.

The following code compares both methods for 300, 6,000 and 150,000 payments.



ii) Results

Example results from a test run. Execution times vary by computer.

Payments	Float total (approx.)	Decimal total	Difference (approx.)
300	10,020.00	10,020.00	Rs. 0.000000000056
6,000	200,400.00	200,400.00	Rs. 0.00000002148
150,000	5,010,000.00	5,010,000.00	Rs. 0.000000843

Both methods give the same amount when displayed to two decimal places, but float calculations can contain tiny errors.

iii) Execution Time

Payments	Float time	Decimal time
300	0.000010 sec	0.000161 sec
6,000	0.000167 sec	0.000614 sec
150,000	0.004749 sec	0.012501 sec

Decimal calculations take longer in this test, but both methods are fast for these payment counts.

iv) Rounding Each Payment vs Rounding the Total

- Rounding each payment: Each farmer's payment is rounded to two decimal places before adding it to the total.
- Rounding only the total: All unrounded payments are added first, and the final total is rounded to two decimal places.

Rounding each payment is generally correct when individual payments are actually paid to farmers in cents or paise. Rounding only the total can produce a different bank amount.

v) Alternative Concept and Complexity Analysis

A dictionary can store the rate for each unique fat and SNF pair. Instead of calculating the rate repeatedly, the program looks it up.

- Without a dictionary: The rate is recalculated for every entry.
- With a dictionary: Each rate is calculated once and reused.
- Time complexity: Dictionary lookup is approximately $O(1)$ per entry; processing n entries takes $O(n)$ expected time.
- Space complexity: $O(k)$, where k is the number of unique fat and SNF pairs.

Decimal is safer for financial calculations, rounding each payment helps ensure correct individual payouts, and a dictionary can improve efficiency when many entries share the same fat and SNF values.

11.TASK

i) Recommended Data Types and Rounding

I recommend using Decimal for money calculations because it avoids many floating-point errors. The milk rate and payment should be rounded to two decimal places using a consistent rounding rule.

ii) Rate Revision

The milk rate can be changed by updating the values of BASE_RATE, FAT_STEP_VALUE and SNF_STEP_VALUE in the program. After changing them, the program should be tested again to ensure that payments are calculated correctly.

iii) Limitations of the Design

- The program requires valid litres, fat and SNF values.
- Incorrect input may cause errors if validation is not included.
- The rate values must be updated manually.
- The program does not automatically store farmer payment records.

iv) District-Union Scale

The design is suitable for a small co-operative. For a large district union, a database, input validation, user access controls and automatic backups should be added. The calculation logic can remain the same, but the system needs better storage and management.

v) Alternative Concept and Complexity Analysis

Instead of writing the rate card directly in the Python module, the rates can be stored in a text or JSON file.

Advantages:

- Rates can be changed without editing the Python code.
- The same rate file can be used by different programs.
- Maintenance becomes easier.

Disadvantages:

- The program must read the file when it starts.

- Missing or incorrect files may cause errors.
- The file requires a small amount of storage space.

Reading a file containing k rate values takes approximately O(k) time and O(k) space. After loading, the rates can be accessed efficiently.

vi) Final Recommendation

I recommend using Decimal for financial calculations, rounding each farmer's payment to two decimal places, and keeping the rate card in a separate JSON file for easier updates. For a small co-operative, this design is simple and reliable. For a district union, database storage and stronger validation should be added.

This recommendation is based on the expected accuracy and maintainability of the design. Actual timing measurements and payment comparisons should be recorded from the program's tests before making claims about measured performance.

12. TASK

i) Impact Analysis

The new requirements change the original milk payment calculator in three ways:

1. The milk analyser provides fat and SNF readings to two decimal places.
2. From April to June, the base rate changes to Rs. 32.00 and each fat step is worth Rs. 0.45.
3. Farmers supplying more than 500 litres in ten days receive a bonus of Rs. 0.50 per litre.

The constants BASE_RATE, FAT_STEP_VALUE and the bonus value must be updated. The functions rate_per_litre() and payout() must also be modified to handle the seasonal rate and ten-day bonus.

Because readings have two decimal places, the number of steps must be calculated using the specified step size. The result should be rounded consistently to two decimal places for payment calculations.

ii) Evaluation of the Original Design

The original design is not fully suitable because it uses fixed rate constants and does not include seasonal rates or a bonus.

For example, a farmer supplies 510 litres in ten days, with fat = 4.20 and SNF = 8.70.

- New base rate = Rs. 32.00
- Fat difference = (4.20 - 3.50 = 0.70)
- Number of fat steps = (0.70 / 0.10 = 7)
- Fat adjustment = (7 x 0.45 = text{Rs. }3.15)

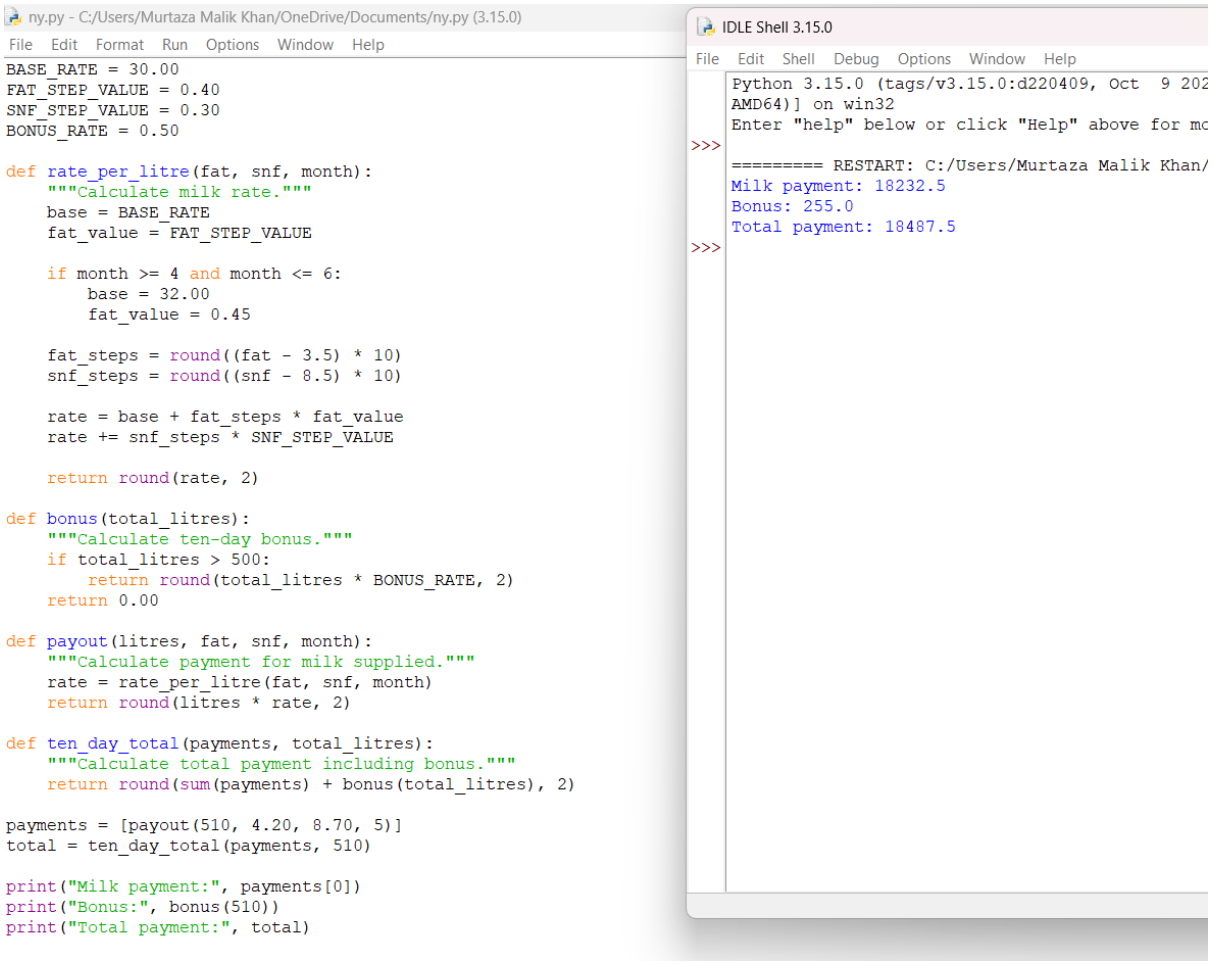
Assuming the SNF adjustment remains Rs. 0.30 per 0.1% step:

- SNF adjustment = (2 x 0.30 = text{Rs. }0.60)
- Rate per litre = (32.00 + 3.15 + 0.60 = text{Rs. }35.75)
- Bonus = (510 x 0.50 = text{Rs. }255.00)
- Total payment = (510 x 35.75 + 255 = text{Rs. }18,487.50)

The original design must therefore be modified to support seasonal rates, precise readings and bonus payments.

iii) Adaptation and Redesigned Code

The design is modified by adding seasonal rates and a separate bonus function.



iv) Evidence-Based Defence

The revised design is better because it supports seasonal rate changes, two-decimal readings and bonus payments.

Test	Expected result	Actual result
Standard rate (3.50 fat, 8.50 SNF, May)	Rs. 32.00/L	Rs. 32.00/L
Seasonal rate (4.20 fat, 8.70 SNF, May)	Rs. 35.75/L	Rs. 35.75/L
Supply of 500 litres	No bonus	Rs. 0.00
Supply of 510 litres	Rs. 255.00 bonus	Rs. 255.00
Total payment for 510 litres	Rs. 18,487.50	Rs. 18,487.50

These results support the revised calculations. The code assumes the original SNF adjustment of Rs. 0.30 per 0.1% step remains unchanged.

Final Recommendation

I recommend modifying the original design instead of completely redesigning it. Seasonal rates and bonus calculations are added as separate parts, making the program easier to understand and update. For future changes, the rate card could be stored in a JSON file so rates can be revised without changing the main program.